The Rust Programming Language

Chapter 4: Understanding Ownership

Ownership is Rust’s most unique feature. It lets Rust make memory safety guarantees without needing a garbage collector.

Contents

4.1 What Is Ownership? page 2
4.2 References and Borrowing page 3
4.2 Mutable and dangling references page 4

4.1 What Is Ownership?

Ownership rules:

1. Each value in Rust has an owner.
2. There can only be one owner at a time.
3. When the owner goes out of scope, the value will be dropped.

The String type:

```
let s = String::from("hello");
s.push_str(", world!");
println!("{s}");
```

A String is allocated on the heap. When s goes out of scope,
Rust calls drop and returns the memory.

4.2 References and Borrowing

A reference lets you refer to a value without taking ownership.

```
fn calculate_length(s: &String) -> usize { s.len() }
```

The rules of references:

1. At any given time, you can have either one mutable reference or any number of immutable references.
2. References must always be valid.

4.2 Mutable and dangling references

Mutable references have one big restriction: if you have a mutable reference to a value, you can have no other references to that value.

```
let r1 = &mut s;
let r2 = &mut s;  // error[E0499]
```

A dangling reference points to memory that was freed. Rust guarantees that references never dangle: the compiler checks that the data lives at least as long as the reference.